Notebook 2: LLM Translation and Evaluation

This notebook demonstrates **pure LLM translation** and **decoupled evaluation**.

Key concepts:

1. **Translation** - LLM converts narrative to config (single prompt, no tools)
2. **Verification** - Run actual portfolio optimization to prove translation works
3. **Evaluation** - Completely separate step: field accuracy + LLM-as-judge

All functions are traced to Langfuse via `@observe()` decorator.

Setup

In [1]:
```python
import os
import json
import warnings
from typing import Dict, Any, List
from dotenv import load_dotenv

load_dotenv()
load_dotenv(dotenv_path='../.env')

# Import TRANSLATION functions
from llm_utils import (
    translate_narrative,
    PortfolioConfigSchema,
    LLMProvider
)

# Import EVALUATION functions (decoupled from translation)
from llm_utils import (
    compute_field_accuracy,
    llm_as_judge,
    evaluate_single,
    flush_langfuse
)

# Import portfolio optimization for verification
from portfolio_optimizer import (
    PortfolioConfig, download_market_data, optimize_portfolio,
    optimize_hrp, backtest_portfolio, get_universe
)

warnings.filterwarnings('ignore')

with open('scenarios.json', 'r') as f:
    SCENARIOS = json.load(f)
```

```
with open('evaluation_dataset.json', 'r') as f:
    EVAL_DATA = json.load(f)

DEFAULT_PROVIDER = LLMProvider.GEMINI

print("Setup complete!")
print(f"Default LLM Provider: {DEFAULT_PROVIDER.value}")
print(f"Loaded {len(SCENARIOS['personas'])} personas")
print(f"Loaded {len(EVAL_DATA['items'])} evaluation items")
```

```
/Users/oleksandrhonchar/.pyenv/versions/3.12.10/lib/python3.12/site-package
s/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter a
nd ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.
html
  from .autonotebook import tqdm as notebook_tqdm
Setup complete!
Default LLM Provider: gemini
Loaded 3 personas
Loaded 20 evaluation items
```

Part 1: Translation

The `translate_narrative()` function:

- Takes an investor narrative as input
- Uses a single LLM call with structured output
- Returns a portfolio configuration JSON
- **No tools, no agents** - pure prompt-based translation

In [2]:
```
# Show the configuration schema
print("Portfolio Configuration Schema:")
schema = PortfolioConfigSchema.model_json_schema()
print(f"Required fields: {schema.get('required', [])}")
print(f"\nField descriptions:")
for field, props in schema.get('properties', {}).items():
    desc = props.get('description', 'N/A')[:60]
    print(f"  {field}: {desc}")
```

```
Portfolio Configuration Schema:
Required fields: ['optimization_target', 'universe', 'time_horizon_years',
'risk_tolerance', 'reasoning']

Field descriptions:
  optimization_target: Optimization objective: min_volatility for low risk,
max_sha
  universe: Asset universe to invest in
  time_horizon_years: Investment horizon in years
  max_position: Maximum weight per position (0.05-1.0), null if not specifie
  risk_tolerance: Risk tolerance level inferred from narrative
  allow_short: Whether short selling is allowed
  target_return: Target return for efficient_return optimization, null other
w
  reasoning: Brief explanation of why these parameters were chosen
```

In [3]:
```python
# Translate Marcus's narrative (aggressive growth)
marcus = SCENARIOS['personas']['marcus_growth']
print(f"PERSONA: {marcus['name']}")
print(f"NARRATIVE: {marcus['narrative']}")
print("\n" + "="*60)
print("TRANSLATING...")

marcus_translation = translate_narrative(
    narrative=marcus['narrative'],
    provider=DEFAULT_PROVIDER,
    session_id="nb2_translation"
)

if marcus_translation['status'] == 'success':
    print("\nTRANSLATION RESULT:")
    print(json.dumps(marcus_translation['config'], indent=2))
else:
    print(f"Error: {marcus_translation['error']}")
```

```
PERSONA: Marcus Johnson
NARRATIVE: I'm Marcus, 28 years old, just started my career as a software en
gineer. I have $50,000 to invest and want to maximize long-term growth. I ca
n tolerate high volatility since I won't need this money for 30+ years. I be
lieve in technology and want concentrated exposure to tech stocks.

============================================================
TRANSLATING...
WARNING: All log messages before absl::InitializeLog() is called are written
to STDERR
I0000 00:00:1768482923.241587 29663147 fork_posix.cc:71] Other threads are c
urrently calling into gRPC, skipping fork() handlers
TRANSLATION RESULT:
{
  "optimization_target": "max_return",
  "universe": "us_tech",
  "time_horizon_years": 30,
  "max_position": null,
  "risk_tolerance": "high",
  "allow_short": false,
  "target_return": null,
  "reasoning": "Marcus is young, has a long time horizon, and wants to maxim
ize growth with high risk tolerance and concentrated tech exposure."
}
```

In [4]:
```python
# Translate Sarah's narrative (conservative)
sarah = SCENARIOS['personas']['sarah_conservative']
print(f"PERSONA: {sarah['name']}")
print(f"NARRATIVE: {sarah['narrative']}")
print("\n" + "="*60)
print("TRANSLATING...")

sarah_translation = translate_narrative(
    narrative=sarah['narrative'],
    provider=DEFAULT_PROVIDER,
    session_id="nb2_translation"
```

```
)

if sarah_translation['status'] == 'success':
    print("\nTRANSLATION RESULT:")
    print(json.dumps(sarah_translation['config'], indent=2))
else:
    print(f"Error: {sarah_translation['error']}")
```

```
PERSONA: Sarah Chen
NARRATIVE: I'm Sarah, 58 years old, and planning to retire in 7 years. I hav
e $500,000 saved and want stable income with capital preservation. I can't a
fford to lose more than 15% of my portfolio. I prefer US-based bond investme
nts and don't want emerging market exposure.

============================================================
TRANSLATING...
TRANSLATION RESULT:
{
  "optimization_target": "min_volatility",
  "universe": "conservative",
  "time_horizon_years": 7,
  "max_position": null,
  "risk_tolerance": "low",
  "allow_short": false,
  "target_return": null,
  "reasoning": "Sarah is nearing retirement and prioritizes capital preserva
tion and stable income with a low risk tolerance and a short time horizon. A
conservative universe and min_volatility optimization target align with thes
e goals."
}
```

In [5]:
```
# Translate Elena's narrative (balanced)
elena = SCENARIOS['personas']['elena_balanced']
print(f"PERSONA: {elena['name']}")
print(f"NARRATIVE: {elena['narrative']}")
print("\n" + "="*60)
print("TRANSLATING...")

elena_translation = translate_narrative(
    narrative=elena['narrative'],
    provider=DEFAULT_PROVIDER,
    session_id="nb2_translation"
)

if elena_translation['status'] == 'success':
    print("\nTRANSLATION RESULT:")
    print(json.dumps(elena_translation['config'], indent=2))
else:
    print(f"Error: {elena_translation['error']}")
```

```
PERSONA: Elena Rodriguez
NARRATIVE: I'm Elena, 38 years old, and I want to build a portfolio for my c
hildren's education fund. My kids are 2 and 5 years old, so I have about 18
years. I have $150,000 to invest with moderate risk tolerance. I'd like glob
al diversification and reasonable risk-adjusted returns. No single position
should exceed 15% of the portfolio.

============================================================
TRANSLATING...
TRANSLATION RESULT:
{
  "optimization_target": "max_sharpe",
  "universe": "global_diversified",
  "time_horizon_years": 18,
  "max_position": 0.15,
  "risk_tolerance": "medium",
  "allow_short": false,
  "target_return": null,
  "reasoning": "Elena has a long time horizon (18 years) and a moderate risk
tolerance. A globally diversified portfolio optimized for maximum Sharpe rat
io provides a balance between risk and return, suitable for long-term growt
h. The 15% max position limit ensures diversification."
}
```

Part 2: Verification via Portfolio Optimization

To prove the translation produces usable configs, we run actual portfolio optimization.

In [6]:

```python
def run_optimization(config: Dict, persona_name: str = "Unknown") -> Dict[st
    """Run portfolio optimization using a translated config."""
    print(f"\n{'='*60}")
    print(f"OPTIMIZING PORTFOLIO FOR: {persona_name}")
    print(f"{'='*60}")

    print(f"\nConfig:")
    print(f"  Universe: {config['universe']}")
    print(f"  Target: {config['optimization_target']}")
    print(f"  Risk tolerance: {config['risk_tolerance']}")

    # Get market data
    universe = config['universe']
    tickers = get_universe(universe)
    print(f"\nDownloading data for {len(tickers)} assets...")

    START_DATE = "2019-01-01"
    END_DATE = "2024-01-01"
    prices = download_market_data(tickers, START_DATE, END_DATE)

    # Create portfolio config
    portfolio_config = PortfolioConfig(
        tickers=list(prices.columns),
        start_date=START_DATE,
        end_date=END_DATE,
        optimization_target=config['optimization_target'],
```

```
        max_position=config.get('max_position'),
        target_return=config.get('target_return'),
        weight_bounds=(-1, 1) if config.get('allow_short') else (0, 1)
    )

    # Optimize
    print("Running optimization...")
    try:
        portfolio = optimize_portfolio(portfolio_config, prices)
    except Exception as e:
        print(f"MVO failed ({e}), using HRP...")
        portfolio = optimize_hrp(prices)

    # Backtest
    backtest = backtest_portfolio(portfolio['weights'], prices)

    # Results
    print(f"\n{'-'*40}")
    print("RESULTS:")
    print(f"{'-'*40}")
    print(f"  Expected Return: {portfolio['expected_return']*100:.2f}%")
    print(f"  Volatility: {portfolio['volatility']*100:.2f}%")
    print(f"  Sharpe Ratio: {portfolio['sharpe_ratio']:.3f}")
    print(f"  Backtest Sharpe: {backtest['sharpe_ratio']:.3f}")
    print(f"  Max Drawdown: {backtest['max_drawdown']*100:.2f}%")

    print(f"\n  Top Holdings:")
    sorted_weights = sorted(portfolio['weights'].items(), key=lambda x: -x[1
    for ticker, weight in sorted_weights[:5]:
        if weight > 0.01:
            print(f"    {ticker}: {weight*100:.1f}%")

    return {"portfolio": portfolio, "backtest": backtest}

print("run_optimization function defined!")
```

```
run_optimization function defined!
```

In [7]:
```
# Verify Marcus's translation works
if marcus_translation['status'] == 'success':
    marcus_portfolio = run_optimization(marcus_translation['config'], marcus
```

```
============================================================
OPTIMIZING PORTFOLIO FOR: Marcus Johnson
============================================================

Config:
  Universe: us_tech
  Target: max_return
  Risk tolerance: high

Downloading data for 15 assets...
YF.download() has changed argument auto_adjust default to True
```

```
Running optimization...
MVO failed (target_volatility required for max_return optimization), using H
RP...

----------------------------------------
RESULTS:
----------------------------------------
  Expected Return: 25.84%
  Volatility: 25.14%
  Sharpe Ratio: 1.028
  Backtest Sharpe: 1.028
  Max Drawdown: -33.78%

  Top Holdings:
    IBM: 17.7%
    ORCL: 12.9%
    CSCO: 10.6%
    MSFT: 6.5%
    GOOG: 6.3%
```

In [8]:
```python
# Verify Sarah's translation works
if sarah_translation['status'] == 'success':
    sarah_portfolio = run_optimization(sarah_translation['config'], sarah['r
```

```
============================================================
OPTIMIZING PORTFOLIO FOR: Sarah Chen
============================================================

Config:
  Universe: conservative
  Target: min_volatility
  Risk tolerance: low

Downloading data for 8 assets...
Running optimization...

----------------------------------------
RESULTS:
----------------------------------------
  Expected Return: 0.27%
  Volatility: 5.67%
  Sharpe Ratio: -0.305
  Backtest Sharpe: 0.081
  Max Drawdown: -18.11%

  Top Holdings:
    GOVT: 49.4%
    MBB: 31.6%
    VMBS: 19.0%
```

In [9]:
```python
# Verify Elena's translation works
if elena_translation['status'] == 'success':
    elena_portfolio = run_optimization(elena_translation['config'], elena['r
```

```
============================================================
OPTIMIZING PORTFOLIO FOR: Elena Rodriguez
============================================================

Config:
  Universe: global_diversified
  Target: max_sharpe
  Risk tolerance: medium

Downloading data for 15 assets...
Running optimization...

----------------------------------------
RESULTS:
----------------------------------------
  Expected Return: 7.84%
  Volatility: 9.25%
  Sharpe Ratio: 0.631
  Backtest Sharpe: 0.932
  Max Drawdown: -15.40%

  Top Holdings:
    AGG: 15.0%
    DBC: 15.0%
    GLD: 15.0%
    GOVT: 15.0%
    SPY: 15.0%
```

Part 3: Evaluation (Decoupled)

Evaluation is **completely separate** from translation:

- `compute_field_accuracy()` - compares predicted vs expected configs
- `llm_as_judge()` - uses LLM to score translation quality
- `evaluate_single()` - runs both evaluations together

Each function has its own Langfuse trace.

In [10]:
```python
# Get a test item from evaluation dataset
test_item = EVAL_DATA['items'][0]
narrative = test_item['input']['narrative']
expected = test_item['expected_output']

print("TEST ITEM:")
print(f"Narrative: {narrative[:100]}...")
print(f"\nExpected config:")
print(json.dumps(expected, indent=2))
```

```
TEST ITEM:
Narrative: I'm Marcus, 28 years old, just started my career as a software en
gineer. I have $50,000 to invest an...

Expected config:
{
  "optimization_target": "max_sharpe",
  "universe": "us_tech",
  "time_horizon_years": 30,
  "risk_tolerance": "high",
  "allow_short": false
}
```

In [11]:
```python
# STEP 1: Translate (separate from evaluation)
print("STEP 1: TRANSLATION")
print("="*60)

translation_result = translate_narrative(
    narrative=narrative,
    provider=DEFAULT_PROVIDER,
    session_id="nb2_eval_demo"
)

if translation_result['status'] == 'success':
    predicted = translation_result['config']
    print("Predicted config:")
    print(json.dumps(predicted, indent=2))
else:
    print(f"Translation failed: {translation_result['error']}")
    predicted = None
```

```
STEP 1: TRANSLATION
============================================================
Predicted config:
{
  "optimization_target": "max_return",
  "universe": "us_tech",
  "time_horizon_years": 30,
  "max_position": null,
  "risk_tolerance": "high",
  "allow_short": false,
  "target_return": null,
  "reasoning": "Marcus is young, has a long time horizon, and wants to maxim
ize growth with high risk tolerance and concentrated tech exposure."
}
```

In [12]:
```python
# STEP 2a: Evaluate with field accuracy (decoupled)
if predicted:
    print("STEP 2a: FIELD ACCURACY EVALUATION")
    print("="*60)

    field_accuracy = compute_field_accuracy(
        predicted=predicted,
        expected=expected,
        session_id="nb2_eval_demo"
    )
```

```
    print(f"Overall accuracy: {field_accuracy['overall_accuracy']:.2%}")
    print("\nField-by-field:")
    for field, score in field_accuracy.items():
        if field != 'overall_accuracy':
            status = "✓" if score == 1.0 else "✗"
            print(f"  {status} {field}: {score}")
```

```
STEP 2a: FIELD ACCURACY EVALUATION
============================================================
Overall accuracy: 80.00%

Field-by-field:
  ✗ optimization_target_match: 0.0
  ✓ universe_match: 1.0
  ✓ risk_tolerance_match: 1.0
  ✓ allow_short_match: 1.0
  ✓ time_horizon_years_match: 1.0
```

In [13]:
```
# STEP 2b: Evaluate with LLM-as-judge (decoupled)
if predicted:
    print("STEP 2b: LLM-AS-JUDGE EVALUATION")
    print("="*60)

    llm_scores = llm_as_judge(
        narrative=narrative,
        predicted=predicted,
        expected=expected,
        provider=DEFAULT_PROVIDER,
        session_id="nb2_eval_demo"
    )

    print("LLM Judge Scores:")
    for key, value in llm_scores.items():
        print(f"  {key}: {value}")
```

```
STEP 2b: LLM-AS-JUDGE EVALUATION
============================================================
LLM Judge Scores:
  optimization_target_score: 6
  universe_score: 10
  risk_assessment_score: 10
  constraints_score: 8
  overall_score: 8.5
  feedback: Optimization target should be max_sharpe, not max_return. The re
st is good.
```

In [14]:
```
# Combined evaluation using evaluate_single()
if predicted:
    print("COMBINED EVALUATION (evaluate_single)")
    print("="*60)

    full_eval = evaluate_single(
        narrative=narrative,
        predicted=predicted,
        expected=expected,
        provider=DEFAULT_PROVIDER,
```

```
        session_id="nb2_eval_demo"
    )

    print(f"\nSummary:")
    print(f"  Field Accuracy: {full_eval['overall_field_accuracy']:.2%}")
    print(f"  LLM Score: {full_eval['overall_llm_score']:.1f}/10")
```

```
COMBINED EVALUATION (evaluate_single)
============================================================
Summary:
  Field Accuracy: 80.00%
  LLM Score: 8.5/10
```

Part 4: Langfuse Experiments (Official SDK Pattern)

Following the Langfuse experiments tutorial:

Workflow:

1. **Create dataset** in Langfuse with evaluation items
2. **Run experiment** using `dataset.run_experiment()`
3. **View results** with `result.format()` and in Langfuse dashboard

Key components:

- **Task function** - `translation_task(item)` → returns output
- **Item-level evaluators** - `field_accuracy_evaluator`, `llm_judge_evaluator`
- **Run-level evaluators** - `avg_accuracy_evaluator`, `avg_llm_score_evaluator`

In [15]:
```
# Step 1: Create a LANGFUSE DATASET with evaluation items
from langfuse import get_client

print("STEP 1: CREATE LANGFUSE DATASET")
print("="*60)

langfuse = get_client()

# Prepare items for the dataset
dataset_items = [
    {
        "input": {"narrative": item['input']['narrative']},
        "expected_output": item['expected_output']
    }
    for item in EVAL_DATA['items'][:5]  # Use first 5 items
]

DATASET_NAME = "portfolio-translation-v3"

# Create the dataset in Langfuse
dataset = langfuse.create_dataset(
```

```
    name=DATASET_NAME,
    description="Portfolio translation evaluation - investor narratives to c
)

# Add items to the dataset
for i, item in enumerate(dataset_items):
    langfuse.create_dataset_item(
        dataset_name=DATASET_NAME,
        input=item["input"],
        expected_output=item["expected_output"],
        metadata={"index": i}
    )
    print(f"  Added item {i+1}/{len(dataset_items)}")

langfuse.flush()
print(f"\nDataset '{DATASET_NAME}' created with {len(dataset_items)} items")
```

```
STEP 1: CREATE LANGFUSE DATASET
============================================================
  Added item 1/5
  Added item 2/5
  Added item 3/5
  Added item 4/5
  Added item 5/5

Dataset 'portfolio-translation-v3' created with 5 items
```

In [16]:

```
# Step 2: Define TASK and EVALUATORS
from langfuse import Evaluation

print("STEP 2: DEFINE TASK AND EVALUATORS")
print("="*60)

# TASK FUNCTION: Takes item, returns output
def translation_task(*, item, **kwargs):
    """Task: Translate narrative to portfolio config."""
    narrative = item.input.get("narrative")
    result = translate_narrative(narrative, provider=DEFAULT_PROVIDER)

    if result["status"] == "success":
        return result["config"]
    return {"error": result.get("error")}

# ITEM-LEVEL EVALUATOR: Field accuracy
def field_accuracy_eval(*, output, expected_output, **kwargs):
    """Evaluator: Compare predicted vs expected fields."""
    if not output or "error" in output:
        return Evaluation(name="field_accuracy", value=0.0, comment="Transla

    matches = 0
    total = 0
    for field in ["optimization_target", "universe", "risk_tolerance", "allo
        if field in expected_output:
            total += 1
            if output.get(field) == expected_output.get(field):
                matches += 1
```

```
    accuracy = matches / total if total > 0 else 0.0
    return Evaluation(name="field_accuracy", value=accuracy, comment=f"{matc

# ITEM-LEVEL EVALUATOR: LLM judge
def llm_judge_eval(*, input, output, expected_output, **kwargs):
    """Evaluator: Use LLM to judge translation quality."""
    if not output or "error" in output:
        return Evaluation(name="llm_judge", value=0.0, comment="Translation

    narrative = input.get("narrative", "")
    scores = llm_as_judge(narrative, output, expected_output, provider=DEFAU

    overall = scores.get("overall_score", 0) / 10.0
    feedback = scores.get("feedback", "")
    return Evaluation(name="llm_judge", value=overall, comment=feedback)

# RUN-LEVEL EVALUATOR: Average accuracy
def avg_accuracy_eval(*, item_results, **kwargs):
    """Run evaluator: Calculate average field accuracy."""
    accuracies = [
        e.value for r in item_results for e in (r.evaluations or [])
        if e.name == "field_accuracy" and e.value is not None
    ]
    if not accuracies:
        return Evaluation(name="avg_field_accuracy", value=None)
    avg = sum(accuracies) / len(accuracies)
    return Evaluation(name="avg_field_accuracy", value=avg, comment=f"Averag

print("Defined:")
print("  • translation_task(item) → config")
print("  • field_accuracy_eval(output, expected_output) → Evaluation")
print("  • llm_judge_eval(input, output, expected_output) → Evaluation")
print("  • avg_accuracy_eval(item_results) → Evaluation")
```

```
STEP 2: DEFINE TASK AND EVALUATORS
============================================================
Defined:
  • translation_task(item) → config
  • field_accuracy_eval(output, expected_output) → Evaluation
  • llm_judge_eval(input, output, expected_output) → Evaluation
  • avg_accuracy_eval(item_results) → Evaluation
```

In [17]:

```
# Step 3: Run EXPERIMENT on the dataset
print("STEP 3: RUN EXPERIMENT")
print("="*60)

# Get the dataset from Langfuse
dataset = langfuse.get_dataset(name=DATASET_NAME)

# Run experiment with task and evaluators
result = dataset.run_experiment(
    name="gemini-baseline",
    description="Baseline evaluation with Gemini 2.0 Flash",
    task=translation_task,                          # Task function
    evaluators=[field_accuracy_eval, llm_judge_eval],  # Item-level evaluato
```

```
    run_evaluators=[avg_accuracy_eval]              # Run-level evaluators
)

# Display results
print("\n" + "="*60)
print("EXPERIMENT RESULTS")
print("="*60)
print(result.format())
```

```
STEP 3: RUN EXPERIMENT
============================================================
============================================================
EXPERIMENT RESULTS
============================================================
Individual Results: Hidden (10 items)\n💡 Set include_item_results=True to v
iew them\n\n──────────────────────────────────────────────────────────\n🧪 Experimen
t: gemini-baseline
📋 Run name: gemini-baseline - 2026-01-15T13:15:33.925190Z - Baseline evalua
tion with Gemini 2.0 Flash\n10 items\nEvaluations:\n  • llm_judge\n  • field
_accuracy\n\nAverage Scores:\n  • llm_judge: 0.948\n  • field_accuracy: 0.92
0\n\nRun Evaluations:\n  • avg_field_accuracy: 0.920\n    ⌚ Average: 92.00%
\n\n🔗 Dataset Run:\n   https://cloud.langfuse.com/project/cmkef3ofc00v2ad07
9i334w09/datasets/cmkfg0fz201zxad06dpgcna33/runs/f7701260-9fa1-4a55-bfca-f4b
72de05e17
```

In [18]:

```
# Step 4: View detailed results
print("STEP 4: DETAILED RESULTS")
print("="*60)

# Run-level evaluations
print("\nRUN-LEVEL EVALUATIONS:")
for eval in result.run_evaluations:
    val = f"{eval.value:.2%}" if eval.value is not None else "N/A"
    print(f"  {eval.name}: {val}")
    if eval.comment:
        print(f"    → {eval.comment}")

# Item-level results
print(f"\nITEM-LEVEL RESULTS:")
for i, item_result in enumerate(result.item_results):
    print(f"\nItem {i+1}:")
    if item_result.output and isinstance(item_result.output, dict):
        opt = item_result.output.get('optimization_target', 'N/A')
        uni = item_result.output.get('universe', 'N/A')
        print(f"  Output: {opt} / {uni}")
    for eval in item_result.evaluations:
        val = f"{eval.value:.2%}" if eval.value is not None else "N/A"
        comment = f" ({eval.comment})" if eval.comment else ""
        print(f"  {eval.name}: {val}{comment}")
```

```
STEP 4: DETAILED RESULTS
============================================================

RUN-LEVEL EVALUATIONS:
  avg_field_accuracy: 92.00%
    → Average: 92.00%

ITEM-LEVEL RESULTS:

Item 1:
  Output: min_volatility / conservative
  field_accuracy: 100.00% (5/5 fields matched)
  llm_judge: 100.00% (Excellent translation. All aspects of the predicted co
nfiguration align perfectly with the investor's narrative.)

Item 2:
  Output: max_return / us_tech
  field_accuracy: 80.00% (4/5 fields matched)
  llm_judge: 90.00% (The universe, risk assessment, and constraints are hand
led well. However, 'max_sharpe' is a better optimization target than 'max_re
turn' given the desire for aggressive growth while considering risk. 'max_re
turn' alone can lead to excessive risk-taking.)

Item 3:
  Output: max_sharpe / global_diversified
  field_accuracy: 100.00% (5/5 fields matched)
  llm_judge: 97.50% (Excellent translation. All parameters are correctly ext
racted and interpreted. The choice of 'max_sharpe' is appropriate for modera
te risk tolerance and long time horizon.)

Item 4:
  Output: min_volatility / conservative
  field_accuracy: 100.00% (5/5 fields matched)
  llm_judge: 95.00% (Excellent translation. The optimization target, univers
e, and risk assessment are perfectly aligned with the narrative. The constra
ints handling is mostly correct, although the absence of explicit constraint
s like 'max_position' is acceptable given the conservative approach. The nar
rative's preference for US-based bonds is implicitly captured by the 'conser
vative' universe.)

Item 5:
  Output: max_return / us_tech
  field_accuracy: 80.00% (4/5 fields matched)
  llm_judge: 90.00% (The universe, risk assessment, and constraints are well
-handled. Optimization target should be max_sharpe instead of max_return for
a more balanced approach, even with high risk tolerance.)

Item 6:
  Output: min_volatility / conservative
  field_accuracy: 100.00% (5/5 fields matched)
  llm_judge: 100.00% (Excellent translation. All aspects of the investor's n
arrative were accurately captured and translated into the portfolio configur
ation.)

Item 7:
  Output: max_return / us_tech
```

```
  field_accuracy: 80.00% (4/5 fields matched)
  llm_judge: 90.00% (The universe, risk assessment, and constraints are hand
led well. However, 'max_sharpe' is a better optimization target than 'max_re
turn' given the desire for aggressive growth while considering risk. 'max_re
turn' alone can lead to excessive risk-taking.)

Item 8:
  Output: max_sharpe / global_diversified
  field_accuracy: 100.00% (5/5 fields matched)
  llm_judge: 97.50% (Excellent translation. All parameters are correctly ext
racted and interpreted. The choice of 'max_sharpe' is appropriate for modera
te risk tolerance and long time horizon.)

Item 9:
  Output: min_volatility / conservative
  field_accuracy: 100.00% (5/5 fields matched)
  llm_judge: 97.50% (Excellent translation. All key aspects of the narrative
were accurately captured. The only minor deduction is due to the missing max
_position and target_return in the expected config, which were present in th
e predicted config. However, the core elements are spot on.)

Item 10:
  Output: max_return / us_tech
  field_accuracy: 80.00% (4/5 fields matched)
  llm_judge: 90.00% (The universe, risk assessment, and constraints are well
-handled. Optimization target should be max_sharpe instead of max_return for
a more balanced approach, even with high risk tolerance.)
```

In [19]:

```python
# Flush and view in Langfuse dashboard
langfuse.flush()

print("="*60)
print("VIEW IN LANGFUSE DASHBOARD")
print("="*60)
print(f"\nDataset: {DATASET_NAME}")
print(f"Experiment: {result.name}")
print("\nNavigation: Langfuse → Datasets → Select dataset → View runs")
print("\nKey pattern from tutorial:")
print("""
result = dataset.run_experiment(
    name="experiment-name",
    task=my_task,                    # def my_task(*, item, **kwargs) -> out
    evaluators=[my_evaluator],       # def my_eval(*, output, expected_outpu
    run_evaluators=[my_run_eval]     # def my_run_eval(*, item_results, **kw
)
""")
```

```
============================================================
VIEW IN LANGFUSE DASHBOARD
============================================================

Dataset: portfolio-translation-v3
Experiment: gemini-baseline

Navigation: Langfuse → Datasets → Select dataset → View runs

Key pattern from tutorial:

result = dataset.run_experiment(
    name="experiment-name",
    task=my_task,                    # def my_task(*, item, **kwargs) -> out
put
    evaluators=[my_evaluator],       # def my_eval(*, output, expected_outpu
t, **kwargs) -> Evaluation
    run_evaluators=[my_run_eval]     # def my_run_eval(*, item_results, **kw
args) -> Evaluation
)
```